

2位解法

Rank	2
Team	Richard_U
Author	Richard_U
Topic ID	549718
Version	Japanese Translation

Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549718>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このコンペを開催したMaastricht UniversityとKaggleに感謝します。また、Discussionと惜しめないアイデア共有を行ったKaggleコミュニティにも感謝します。特に [yunsuxiaozi](#) の [MCTS Starter](#) から、[LudRules](#) よりARI、McAlpine EFLAW、CLRIスコアを計算するアイデアを採用しました。ありがとうございました！

多くの人と同じく大きなshake-upを予想しており、自分の解法が上位に入るとはまったく思っていませんでした。そのため今朝、PublicとPrivateで上位3位が変わらなかったことに驚きました。

更新： [提出Notebook](#)／[モデルdataset](#)と[offline学習の再現手順](#)

概要

私の解法は、拍子抜けするほど単純でした。

- 伝統的な5分割CV学習
- offlineで生成した追加学習データなし
- [LudRules](#) 由来のtext score以外、凝った特徴量設計なし
- 情報のない特徴量・重複特徴量と2つのrule-set特徴量を落とす以外、特徴量選択なし

性能上の大きな突破口は明らかにデータ拡張でした。[3位解法](#) とまったく同じaugmentationです。学習データを見れば自然な方法であり、多くの人が独立に同じ拡張を使ったはずです。

他の報告とは対照的に、第2学習の特徴量としてOOF予測を使う実験はまったく成功しませんでした。ただし時間をあまり使っておらず、単なるtypoで壊していた可能性もあります。そのためモデルは通常の1回の学習・予測構造です。

最終モデルはboosting 4個とNN 3個、計7モデルの重み付きアンサンブルでした。さらに、学習seedだけが異なる同一構造のモデルセット6組を等重みで混ぜました。

データ準備とCV split

- 重複特徴量と情報のない特徴量を削除。
- [LudRules](#) からARI、McAlpine EFLAW、CLRIを計算。
- [LudRules](#) 由来groupでデータを並べ替え。TF-IDF vectorizationとKMeans clusteringにより、元の1,373 rulesetを約1,270 group（seedで多少変化）へまとめました。非常に似たゲームをCV split上で同一ゲームとして扱い、学習データ外への汎化性が高いCV推定を作る狙いです。実際に効いたかは不明ですが、筋が通っており悪影響はなさそうでした。
- [LudRules](#)、[EnglishRules](#)、[GameRulesetName](#)、[Id](#) を削除。
- [agent1](#) と [agent2](#) を交換し、[AdvantageP1](#) を 1-[AdvantageP1](#) に、[utility_agent1](#) を -[utility_agent1](#) にして拡張。拡張由来の行をモデルに知らせるdummy特徴量も加えました。直感に反しますがCVが少し改善しました。推論時にも同じ拡張を使い、各モデルから元行と拡張行の2予測を作り、後者は-1倍して元の向きへ戻しました。
- [agent](#)のカテゴリ列をone-hot dummy encoding（最頻値をbaseline）。

- LudRules の一意なruleset groupをdummy encoding。この特徴量は一部boostingモデルだけで使用。学習外では全て0になるため、NNには有益でなく使いませんでした。
- NNの学習・推論前に連続変数を標準正規分布へ写像。

モデル構造と学習

7モデルのアンサンブル：

1. CatBoost (ruleset identifier dummyなし)
2. CatBoost (ruleset identifier dummyあり)
3. LightGBM (dummyなし)
4. LightGBM (dummyあり)
5. MLP (dummyなし)
6. より大きなMLP (dummyなし)
7. Auto-Encoderに続く MLP (dummyなし)

モデル2と4へ全ruleset group dummyを加えた点を除き、すべて同じ特徴量を使用しました。

各モデルの予測をtarget範囲 `[-1, 1]` にclipし、OLS回帰で求めたCV重みにより結合しました。NNの拡張行予測は明確な利点がなく、この段階では除外しました。意外にも、CVは大きなMLP (model 6) とAE NN (model 7) へ明確な負重みを付けました。正重みが期待される場面で負重みを使うことには懐疑的で、本来はモデル構造・仕様に改善余地がある兆候だと考えます。しかし制限時間内に改善できず、CVとPublic LBの双方に有益だったため負重みも含めました。過学習したモデルの負重みが学習外汎化を改善するという仮説も、完全に不合理ではないと思いました。

ensemble予測を再びclipし、CV OLS回帰でscaleしました。過学習が怖かったものの、CV・Public LB双方を改善したため維持しました。

同一構造でseedの異なる6セットを学習しました。seedは半ランダムで、CVが低すぎる／高すぎるものは手作業で除外しました。中庸なCVの方がPublic LBへ汎化したためです。最後に6セットの結合予測を再びtarget範囲へclipしました。

Public LBへの直接的な最終調整

上の方法で主にCVベースの提出を作りました。さらに、Leaderboardへ全面的にoverfitしたと思える提出も作りました。違いは2点です。

1. 除外していたNNの拡張データ予測を、Public LBへ直接fitした負重みで追加。
2. 全モデル予測を等重みとした予測セットを作り、これもPublic LBへfitした負重みで追加。

典型的でほとんど自暴自棄なoverfitに見え、Privateで勝つ選択肢ではないと思っていました。しかし結果は次のとおりでした。

戦略	Private	Public
主にCVベース	0.42324	0.41779

戦略	Private	Public
Public LBへ全面overfit	0.41996	0.41429

幸運にも予想は外れ、このコンペではLeaderboard overfitが報われました。主催者の[終了後投稿](#)によれば、一部 rulesetがPublicとPrivateの両テストに現れたことも説明の一部です。両方に同じゲームがあるとは想定していませんでしたが、Public LBに整合した提出が報われるのは明らかです。結果には恐縮しており、2位は実力より運の方が大きかったと正直に思います。

改めて、ありがとうございました。